TRADEFORGE

Design Pattern Lab Project — Viva & Presentation Guide

Java 11 Desktop Application | Java Swing GUI | MVC Architecture | 10 GoF Design Patterns | JDBC + MySQL | Maven Build

1. Executive Summary & Architecture

TradeForge is an academic stock trading simulation built using **Java 11**, **Java Swing**, **MVC Architecture**, **JDBC**, and **MySQL**. The application demonstrates exactly **10 Gang of Four (GoF) design patterns** working in an integrated, authentic desktop workflow.

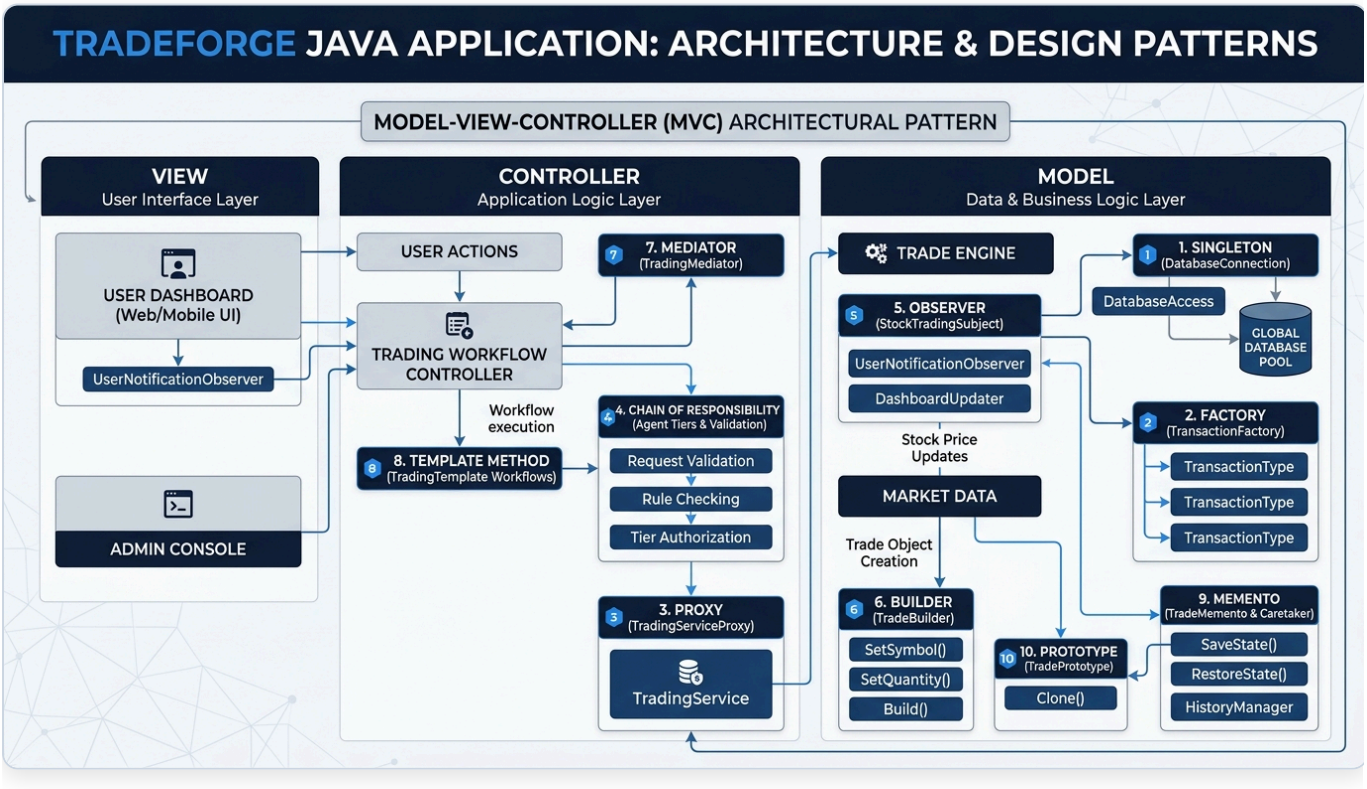


Figure 1: Complete TradeForge Architecture — MVC Layers & 10 Design Patterns Mapping

2. Complete Design Patterns Mapping Table

#	Pattern	Type	Primary Classes	Concrete Responsibility in TradeForge
1	**Singleton**	Creational	`DatabaseConnection`	Provides a shared, thread-safe JDBC connection manager across the application lifecycle to avoid memory leaks.
2	**Factory**	Creational	`TransactionFactory`, `BuyTransaction`, `SellTransaction`	Instantiates BUY or SELL transaction objects to calculate 2% brokerage charges and net totals dynamically.
3	**Proxy**	Structural	`TradingServiceProxy`, `RealTradingService`	Protection proxy: verifies user authentication and ownership before delegating trade execution.
4	**Chain of Resp.**	Behavioral	`Tier1AgentHandler`, `Tier2AgentHandler`, `Tier3AgentHandler`	Routes orders to broker agents based on quantity: 1–500 (Agent 1), 501–700 (Agent 2), >700 (Agent 3). Plus a validation chain.
5	**Observer**	Behavioral	`StockTradingSubject`, `UserNotificationObserver`	Broadcasts trade completion events in real time to update the dashboard's top notification banner.
6	**Builder**	Creational	`TradeBuilder`	Fluent, step-by-step assembly of `TradingRequest` objects with strict field validation before execution.
7	**Mediator**	Behavioral	`TradingMediator`	Central hub that coordinates the Controller, Proxy, Chains, Template Method, DAOs, and Observers, achieving loose coupling.
8	**Template Method**	Behavioral	`TradingTemplate`, `BuyTradingWorkflow`, `SellTradingWorkflow`	Enforces an invariant 5-step trade sequence (validate → check → execute → save → notify) with customized hooks inside an ACID transaction.
9	**Memento**	Behavioral	`TradeMemento`, `TradeCaretaker`	Captures and restores UI trade draft form values (Save Draft and Undo/Restore) without altering database state.
10	**Prototype**	Creational	`TradePrototype`, `TradingRequest`	Deep-clones trade requests (Duplicate Trade) to create an independent editable draft without immediate database writes.

3. Presentation Script for Mam (Step-by-Step Live Demo)

Step 1: Introduction & Authentication (Singleton Pattern)

> **Action: Launch application (run.bat) & show Login screen.**
>
> *"Good morning/afternoon Mam. Today I am presenting **TradeForge**, a desktop stock trading simulation built using Java 11, Swing, MVC, JDBC, and MySQL. When launching the app, the **Singleton Pattern** initializes via `DatabaseConnection`, managing a single thread-safe connection to MySQL to avoid socket leaks. Credentials submitted on the login screen are processed by `LoginController` (MVC Controller layer) without hardcoding SQL in the GUI."*

Step 2: Dashboard Overview (MVC Architecture)

> **Action: Log in with 'anshul' / '1234' and showcase the Dashboard.**
>
> *"Once authenticated, the MVC View layer (`DashboardFrame`) loads:*
>
> - *Available Account Balance (₹50,000.00).*
> - *Market Stock Catalog (`StocksPanel`).*
> - *User Portfolio Holdings (`PortfolioPanel`).*
> - *Interactive Trading Desk (`TradePanel`).*
> - *Completed Transactions Audit Log (`TransactionHistoryPanel`).*
> - *Real-time Observer Notification Banner at the top.*
>
> "

Step 3: Chain of Responsibility (Dynamic Broker Agent Assignment)

> **Action: Switch to Trading Desk tab. Change the Quantity input:**
>
> *"Mam, here we demonstrate the **Chain of Responsibility Pattern**. Observe the dynamic Assigned Agent badge:*
>
> - *When quantity is **100** (1 to 500 stocks): handled by **Agent 1 - Rahul Sharma (Tier 1)**.*
> - *When quantity is changed to **600** (501 to 700 stocks): `Tier1AgentHandler` delegates down the chain to **Agent 2 - Priya Patel (Tier 2)**.*
> - *When quantity exceeds **700** (e.g., 850): delegates to **Agent 3 - Amit Verma (Tier 3)**.*
>
> *In addition, a validation chain verifies account status, stock availability, and sufficient balance before execution."*

Step 4: Factory Pattern (BUY vs. SELL Fee Calculations)

> **Action: Toggle between BUY and SELL radio buttons.**
>
> *"Here we demonstrate the **Factory Pattern** (`TransactionFactory`). When BUY is selected, `BuyTransaction` adds a 2% brokerage fee to the total cost. When SELL is selected, `SellTransaction` deducts the 2% fee from the proceeds. The GUI contains no business math; it simply invokes the Factory product."*

Step 5: Memento Pattern (Save Draft State & Undo)

> **Action: Select INFY, BUY, 50. Click 'Save Draft (Memento)'. Change to WIPRO, SELL, 200. Click 'Undo / Restore (Memento)'.**
>
> *"This is the **Memento Pattern**. `TradeMemento` captures a snapshot of current form inputs and stores it in `TradeCaretaker`. When we alter the inputs and click 'Undo / Restore', the form values instantly revert to the saved snapshot. This restores the UI draft state without rolling back committed database transactions."*

Step 6: Prototype Pattern (Duplicate Trade)

> **Action: Go to Market Stocks or History, select a row, and click 'Duplicate Selected Trade (Prototype)'.**
>
> *"This demonstrates the **Prototype Pattern**. `TradingRequest` implements `TradePrototype` to clone values from an existing holding or transaction record. It creates an independent, editable draft without inserting an immediate duplicate record in the database."*

Step 7: Trade Execution Flow (Builder, Mediator, Proxy, Template, Observer)

> **Action: In Trading Desk, click 'Execute Order via TradeBuilder & Mediator'.**
>
> *"When executing the order, 5 patterns execute in harmony:*
>
> 1. ***Builder (`TradeBuilder`)**: Validates and constructs the `TradingRequest`.*
> 2. ***Mediator (`TradingMediator`)**: Decouples the Controller from backend subsystems.*
> 3. ***Proxy (`TradingServiceProxy`)**: Verifies security and ownership before forwarding to `RealTradingService`.*
> 4. ***Template Method (`TradingTemplate`)**: Runs the fixed 5-step trade sequence inside an ACID JDBC transaction.*
> 5. ***Observer (`StockTradingSubject`)**: Notifies `UserNotificationObserver` upon success, updating the top banner in real time."*

4. Top Viva Questions & Model Answers

Q1: Why did you use the Mediator pattern instead of having the Controller coordinate everything?

Answer: In a trading system, if the Controller directly invoked the Proxy, Builder, Factory, DAOs, Chain handlers, and Observers, it would become an overly complex "God Object" with high coupling. The `TradingMediator` centralizes this coordination so the Controller only interacts with a single, high-level interface.

Q2: How does your Proxy Pattern provide protection?

Answer: `TradingServiceProxy` implements `TradingService` and acts as a security barrier. It checks: (1) user is authenticated (`userId > 0`), (2) caller owns the request, and (3) request is not null. It rejects invalid requests with a `SecurityException` before any database queries execute.

Q3: What is the difference between Prototype and Memento in your app?

Answer: Prototype creates a *new, independent clone* of an object with identical values for cloning or duplication. **Memento** captures an *internal state snapshot* of an existing object so it can be restored to that exact previous state later via Undo.

Q4: Why is the template method in `TradingTemplate` declared `final`?

Answer: Because the Template Method pattern requires the core algorithm skeleton (validate → check availability → execute operation → save transaction → notify observers) to remain invariant. Declaring `processTrade(...)` as `final` prevents subclasses from altering the execution order, while allowing them to customize hook steps.

Q5: How does your system maintain database consistency during trades?

Answer: Inside `RealTradingService`, we disable auto-commit (`connection.setAutoCommit(false)`). All four database mutations—user balance update, stock inventory update, portfolio holding update, and transaction record insertion—execute in a single JDBC transaction. If any error occurs, `connection.rollback()` is triggered, preserving ACID guarantees.

Q6: What are the exact Chain of Responsibility thresholds for Agent Assignment?

Answer:

- **1 to 500 stocks:** Agent 1 (Rahul Sharma — Tier 1)
- **501 to 700 stocks:** Agent 2 (Priya Patel — Tier 2)
- **More than 700 stocks:** Agent 3 (Amit Verma — Tier 3)

Each tier handler checks the condition; if it falls outside its range, it delegates to the next handler in the chain.

5. Build & Verification Status

The project is fully compiled and tested using Apache Maven and JUnit 5:

- **Compilation:** `mvn compile` — 51 source files compiled with 0 errors.
- **Unit Tests:** `mvn test` — 9 comprehensive test cases covering all 10 patterns passed with **0 failures, 0 errors**.
- **Executable Package:** Standalone executable JAR built at `target/tradeforge.jar`.
